Aim- Design a distributed application using RMI for remote computation where client submits two strings to the server and server returns the concatenation of the given strings.

Introduction:

The application allows clients to submit integer values to a server, which then calculates the factorial of the provided integer and returns the result to the client program. The use of RPC facilitates seamless communication between the client and server, enabling efficient and scalable remote computation.

Distributed computing has become essential for handling complex tasks that require significant computational power. Remote Procedure Calls (RPC) provide a mechanism for executing procedures or functions on a remote server, allowing clients to offload computation and receive results. In this design, we focus on a distributed application where clients can submit integer values to a server for the calculation of factorial.

Architecture Overview:

The application consists of two main components: the client and the server. The client initiates the RPC by sending a request to the server with an integer value. The server receives the request, computes the factorial, and returns the result to the client. The communication between the client and server is facilitated through the use of an RPC framework.

RPC Framework Selection:

Selecting an appropriate RPC framework is crucial for the success of the distributed application. Popular choices include gRPC, Apache Thrift, and Java RMI. The chosen framework should support seamless communication, data serialization, and provide language-agnostic compatibility.

Communication Protocol

Define the communication protocol between the client and server. Specify the format of the request and response messages. Consider using a standardized data interchange format such as JSON or Protocol Buffers to ensure interoperability.

Server-Side Implementation

On the server side, implement the RPC endpoint that handles incoming requests. Define a function to calculate the factorial based on the received integer value. Ensure proper error handling and validation to handle edge cases and prevent potential issues.

Client-Side Implementation

The client initiates the RPC by sending a request to the server with the integer value. Implement the client-side code to handle the RPC communication. Properly handle exceptions and timeouts to ensure robustness in the face of network issues.

Security Considerations

Implement secure communication channels between the client and server to protect against potential threats. Use encryption protocols such as TLS to secure data transmission and authenticate both ends of the communication.

Scalability and Load Balancing

Consider implementing mechanisms for scalability, such as load balancing across multiple servers. This ensures that the application can handle a growing number of clients and distribute the computation load effectively.

Testing and Quality Assurance

Thoroughly test the application to ensure correctness and reliability. Implement unit tests for individual components and end-to-end tests to validate the entire system. Consider edge cases, concurrency issues, and network failures during testing.

Conclusion

This design provides a blueprint for building a distributed application using RPC for remote computation of factorials. By carefully considering architecture, communication protocols, security measures, and scalability, the resulting application can efficiently handle remote factorial calculations, demonstrating the power and flexibility of distributed computing with RPC.